La presentación del documento es bastante mala y ocurrió lo que justamente mencioné en clase que me detenía sobre aceptar soluciones hechas a mano. Era posible tomar fotos pero las fotos debían ser editadas con alguna herramienta para quitar las partes como las argollas del cuaderno y para dar realce. Además, la organización en las imágenes es perversa y ni siquiera hay título ni nombre ni nada.

Aunque no debería admitir un trabajo con tan mala presentación, por esta vez lo pasaré pero debo penalizar esto en la calificación.

1. N = 5, v = 3, ac = 1, i = 0, ans = [ ]

Primera Iteración: ans = [1], ac = 3, i = 1

Segunda Iteración: ans = [1,3], ac = 9, i = 2

Tercera Iteración: ans = [1,3,9], ac = 27, i = 3

Cuarta Iteración ans = [1,3,9,27], ac = 81, i = 4

Quinta Iteración ans = [1,3,9,27,81], ac = 243, i = 5

Sexta Iteración ans = [1,3,9,27,81,243], ac = 729, i = 6

```
1 def algoritmo1(N,V):
2     ac=1
3     ans=[]
4     i=0
5     while i<=N:
6         ans.append(ac)
7         ac=ac*V
8         i=i+1
9     return ans
```

* Con esto podemos ver que este codigo calcula las N potencias de un numero v y la agrega a una lista.

Ahora que miramos lo que calcula hagamos el análisis de correctitud

- Problema de calcular Potencias

Esta notación no está bien, debería ser N E Nat.

Entrada: Un $\boxed{N \geq 0 \in \mathbb{N}}$ y un número v.

Salida: $\boxed{\text{Arreglo } A[1 \ldots VN]}$

Esta notación no es apropiada ya que hay no corresponde a ninguna de las dos formas de expresar arreglos mencionadas en clase:
- Arreglo A[0 .. N + 1) tal que ...
- A = [a0, a1, ..., a_{N}] tal que ...

* Invariantes:

$I_0: 0 \leq i \leq N+1$, $I_1: ac = v^{i-1}$, $I_2: \boxed{ans = [v^0, v^1, \ldots, v^{i-1}]}$

Esto no está del todo bien ya que v^{-1} tiene un valor numérico posible y por eso no es del todo clara la notación.

* Teorema: Los Invariantes I_0, I_1 y I_2 se cumplen.

* Demostración: Se procede demostrando la validez de los invariantes para la inicialización y estabilidad.

* Inicialización: De acuerdo a las lineas 2-4 tenemos que $i=0$, $ac=1$, $ans[]$.

- Para el Invariante I_0 tenemos que:

$$0 \leq i \leq N+1$$

$$0 \leq 0 \leq N+1 \checkmark$$

- Para el Invariante I_1 se tiene que:

$$ac = v^i, \quad ac = v^0$$

sabemos que v es un número y cualquier número elevado a la 0 da 1. Por ende

$ac = 1$ ✓

- Para el Invariante I_2 se tiene que:

$ans = [v^0, v^1, \ldots, v^{i-1}]$, tenemos que $i=0$ al momento de la inicialización, por lo que iría hasta v^{-1} sin embargo i no puede ser menor a cero por lo que ese elemento no puede existir entonces

$ans = [\,]$

- Por lo tanto las Invariantes I_0, I_1 e I_2 se cumplen para la Inicialización

* Estabilidad: Consideramos una Iteración arbitraria $i=j$, asumimos que antes de ejecutarla las Invariantes se cumplen, entonces las expresiones:

$$0 \leq j \leq N+1 \checkmark, \quad ac = v^j \checkmark, \quad ans = [v^0, v^1, \ldots, v^{j-1}] \checkmark$$

Son verdaderas. Ahora tratamos de demostrar que después de esta Iteración se siguen cumpliendo

- $\boxed{0 \leq j+1 \leq N+1,\ ac = v^{j+1},\ ans = [v^0, v^1, \ldots, v^j]}$???

Empezemos con el primer Invariante. i pasa a valer $j+1$ y

$$0 \leq j \leq N+1$$

se tiene que cumplir ya que $j < N+1$ o de lo contrario no se hubiera entrado al ciclo.

En la línea 6 tenemos

ans.append(ac)

En este caso $ac = V^j$ ya que asumimos que antes de este ciclo las Invariantes se cumple y el valor de este se modificó una línea después de agregarlo, tenemos que antes del ciclo $ans = [V^0, V^1, \ldots, V^{j-1}]$ y después de esto $ans = [V^0, V^1, \ldots, V^{j-1}, V^j]$, por lo que esta invariante también se cumple.

Por último en la línea 7 tenemos que

$$ac = ac * V$$

nuestro valor anterior de ac es

$$ac = V^j$$

y después de ejecutar esta línea tenemos que

$$ac = V^j * V$$

Por leyes de la matemática

$$ac = V^{j+1}$$

Finalmente i seguirá incrementando hasta tomar el valor de $N+1$ y se acabará el ciclo, en este caso si $i = N+1$

por la estabilidad del invariante I2

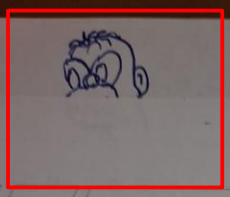
$$\boxed{ans = [V^0, V^1, \ldots, V^N]}$$, lo que coincide con la Poscondición

Esto no es exactamente la poscondición pero el problema está en que la poscondición no está bien.ß

* Teorema: la invocación algoritmo1(N,V) devuelve un arreglo de las N potencias de V

Para cualquier N y v

* Demostración: es trivial a partir de la correctitud de los Invariantes I_0, I_1 e I_2.



?????

$10^0, 10^1$

2. def algoritmo2( ]val,]pos):

```
    i=0
    j=0
    ans=0
    while i < len(lpos):
        if j < lpos[i]:
            j+=i
        else
            ans = ans*10 + lval[j]
            j+=1
            i+=1
    return ans
```

$\sum_{x}^{i}$

lval = [1, 6, 8, 9], lpos = [0, 1, 3] len(lpos) = 3 i=0, j=0

- Primera Iteración: i=1 j=1, ans=1
- Segunda Iteración: i=2 j=2 ans=16
- Tercera Iteración: i=2 j=3 ans=16
- Cuarta Iteración: i=3 j=4, ans=169

y así acaba el ciclo. Básicamente el algoritmo extrae elementos de lval en las posiciones especificadas por P y los concatena.

Entrada: A[0...N] y B[0...M] donde B[M] <= N siendo B un arreglo ordenado ascendentemente.

Salida: $\sum_{N=0}^{M} 10^{i-1} + A[B[i-1]]$

Según esto el tamaño de los arreglos es N + 1 y M + 1

Esto significa que el elemento en la posición M es menor o igual a N

Esto está mal ya que se usa como índice de la sumatoria N en lugar de i pero incluso haciendo ese ajuste con esta expresión no se obtiene el resultado que retorna el agoritmo

1000 ???

A 818 ?????

* Invariantes:

??? $I_0: 0 \leq i \leq M$ $I_1: 0 \leq j \leq B[i]+1$ $I_2: ans = \sum_{N=0}^{i-1} 10^{i+1} + A[B[i-1]]$

Al igual que con la poscondición esta expresión no es correcta.

* Inicialización

De acuerdo a las lineas 2 y 3, $i=0$ y $j=0$, por lo que tenemos que

$I_0: 0 \leq i \leq M$ $I_1: 0 \leq j \leq B[i]+1$

$0 \leq 0 \leq M$ $0 \leq 0 \leq B[i]+1$

Donde tenemos que tanto I_0 como I_1 se cumplen ya que para I_0, M siendo el tamaño del arreglo B como minimo será 0 y $0 \leq 0 \leq 0$ y para I_1 tenemos la misma logica. Ahora tenemos que para I_2 como actualmente no hemos empezado a iterar, $i-1=-1$, entonces la sumatoria no se va a iniciar, por ende el valor será 0 que es el valor con el cual inicializamos ans

Esto está mal dado que la sumatoria planteada no es correcta.

* Estabilidad

Consideramos una iteración arbitraria $i=k$ y $j=x$, suponemos que las invariantes se cumplen en este caso.

$I_0: 0 \leq k \leq M$ $I_1: 0 \leq j \leq B[k]+1$ $I_2: ans = \sum_{N=0}^{k-1} 10^{k-1} + A[B[k-1]]$

Ahora demostremos que se cumplen para la iteración siguiente a esta

Desde la linea 6-11, tenemos dos posibilidades, verifiquemos invariantes para ambos

- 1ra posibilidad: $j < 1[pos[i]]$ y en consecuencia se ejecuta la linea 7.

$j+=1$, $x+=1 = x+1$

En este caso no se modifica ni i ni ans.

Las invariantes se siguen cumpliendo ya que como ni, i ni ans se modifican las invariantes que suponemos correctas queda iguales (I_0 e I_2) Para el caso de I_1 tenemos

$$0 \leq x+1 \leq B[k]+1$$

Esto debido a que si entro al condicional es porque $x < B[k]+1$. Por lo que al aumentar x en 1 el valor máximo que podría tomar es quedar igual a $B[k]+1$ lo que cumple la invariante.

- Para cualquier otro caso se ejecutan las líneas 9-11, donde:

  ans = ans * 10 + lval[j] , i += 1 , j += 1

Después de ejecutar estas líneas tenemos que $i = k+1$, $\cancel{j = x+1}$.

y las invariantes a probar son

$I_0: 0 \leq k+1 \leq M$ $I_1: 0 \leq x+1 \leq B[k+1]+1$ $\boxed{I_2: \sum_{N=0}^{k} 10^k + A[B[k]]}$

I_0 se cumple ya que si se entró al ciclo $k < M$ y al aumentarlo en 1 su mayor valor podría ser M. Para I_1 tenemos que $x \geq B[k]+1$. Sin embargo como k aumentó ahora apunta a un valor mayor teniendo en cuenta que la lista está ordenada ascendentemente por lo que en este caso $x+1$ podría ser mayor a $B[k+1]$ pero como mucho podrá igualar $B[k+1]+1$, lo que cumple el invariante.

Para I_2 tenemos que si la anterior invariante se cumplió entonces 10^k significa que aumentamos en una potencia de 10 (ans * 10) y sumamos $A[B[k]]$, lo que hay en el arreglo A en la posición k, osea en la siguiente posición del ciclo anterior y esto representa (+ lval[j])

debido a esto tenemos que

$$\sum_{k=0}^{j} 10^{k} + A[B[k]] = ans * 10 + A[B[j]] = ans$$

lo que cumple la invariante.

Finalmente i seguirá incrementando hasta alcanzar el valor de M y

~~Se detendrá el ciclo, en ese caso $i = M$, $j = B[M-1]+1$, y~~

$$ans = \sum_{k=0}^{M} 10^{M-1} + A[B[M-1]] = \sum_{k=0}^{M} 10^{i-1} + A[B[i-1]]$$

lo que cumple la salida.

<u>Teorema</u>: La invocación algoritmo2(A,B) para cualquier A y un arreglo B ordenado cuyo último elemento sea menor o igual a len(A)-1 dará la concatenación de A en la posición de los elementos en B.

<u>Demostración</u>: Es trivial a partir de la corrictitud de los invariantes I_0, I_1 e I_2

No es trivial ya que la demostración de la correctitud de los invariantes está mal.

```
def algoritmo3 (A):
    m=None
    r=0
    j= len(A)-1
    while j>=0:
        i,ac=0,0
        while i<j:
            if A[i] <= A[j]:
                ac = ac+1
            i = i+1
        if ac>r:
            r=ac
            m= A[j]
        j=j-1
    return m
```

A[3,1,6,2] ; j = len(A)-1 = 3

Primera Iteración: Se entra al segundo ciclo con i, ac = 0,0

Segundo Ciclo:

Primera Iteración: A[i]=3, A[j]=2 . 3<=2. No. i=1

Segunda Iteración: 1<=2. Sí, ac=1, i=2

Tercera Iteración: 6 <=2. No, i=3 y no entramos más al ciclo

Volvemos al Primer ciclo:

ac>r, 1>0. Sí, r=1, m=2

j=2

Segunda Iteración

Segundo ciclo:

Primera Iteración: 3<6, sí, ac=1, i=1

Segunda Iteración: 1<6, sí, ac=2, i=2

Volvemos al primer ciclo:

2>1, sí. r=2, m=6

j=1

Esto debería expresarse de manera formal o al menos intentarlo.

* hasta esta parte podemos apreciar que el algoritmo calcula con más números menores o iguales a este en posiciones anteriores de la lista

* Entradas: El algoritmo recibe un arreglo A[0... N) de números, N > 0.

* Salida: Un número v, siendo v el número con más números menores o iguales a el en posiciones anteriores del arreglo.

* Invariantes:

$I_0 = -1 \leq j \leq N-1$ $I_1 = 0 \leq i \leq j$ $I_2 = r$ siempre va a ser el mayor número de veces que se encuentra un elemento menor o igual a un elemento de A.

* Inicialización

Tenemos que para la inicialización, $j = len(A)-1$, $i=0$ y $r=0$. En donde se cumple la invariante I_0 ya que $len(A)-1 = N-1$ y $j \leq N-1$, se cumple I_1 ya que $0 \leq 0$ y por ultimo I_2 ya que aún no hemos encontrado ningún número menor a $A[j]$ por lo que tiene sentido que tenga un valor de 0.

* Estabilidad: Vamos a asumir una iteración k en donde se cumplen los invariantes I_0, I_1 e I_2. Tenemos que $j=k$ e $i=x$. Por ende tenemos

R

$I_0 = -1 \leq K \leq N-1$, $I_1 = 0 \leq x \leq j$ y $I_2 = r$ tendra el valor anteriormente dicho con los valores estudiados hasta k. Ahora probemos que el ciclo interno este correcto para la proxima iteración y luego probamos lo mismo con el ciclo externo, con $K+1$ y $x+1$.

La función algoritmo3 tiene un ciclo anidado y debería hacerse una doble demostración. Primero se debe demostrar los invariantes del ciclo interno y luego se debe demostrar los invariantes del ciclo externo.

* Para el ciclo interno tenemos dos posibles casos:

1. Si $A[x+1] <= A[K-1]$, entonces se ejecuta la linea 19 del ciclo donde $ac = ac+1$.

Para este caso tenemos que nuestras invariantes se cumplen ya que para entrar a los ciclos $x+1 \leq j$ (no usamos K ya que la comparación se hace con el valor original de j) y $K-1 \geq -1$, Teniendo esto en cuenta cumplimos las Invariantes I_0 e I_1, la Invariante I_2 tenemos que para el ciclo anterior era correcta y como aún no se modifica el valor de r suponemos que sigue siendo cierto. Comprobamos que para la condición se cumplen, luego simplemente i aumenta en 1 y se siguen haciendo las comparaciones hasta que eventualmente i alcanza el valor de j y sigue con el ciclo externo, ahora probemos la validez de este.

Para el ciclo externo también tenemos una posibilidad y es que $ac > r$, si se cumple esto se ejecutan las lineas 12-13 del codigo donde se actualiza r y m.

$r = ac$

$m = A[j]$

Probemos que se cumplan las Invariantes. Tenemos para I_0 e I_1 que estas ya se cumplen por lo demostrado en el ciclo pasado, al final del ciclo interno $i = j$, osea $x+1 = k+1$, lo que cumple

$0 \leq x+1 \leq k+1$

Para el caso de I_0 como $k+1$ no cambia no es necesario volverla a probar.

Ahora para I_2 tenemos que en caso de que se cumpla el condicional $r = ac$ lo que sigue cumpliendo el Invariante ya que el elemento $k+1$ se fue comparando con cada elemento de $x+1$ hasta que este alcanzó el valor de $k+1$, es decir, comparo el elemento con cada uno de los anteriores a este y aumentaba en 1 el valor de ac cada vez que habia uno menor o igual a este, tenemos que para el ciclo anterior r ya era la cantidad de veces que se encontró un número menor o igual a $A[k]$ por ende si se entra al condicional y actualiza el valor de r es porque el valor $A[k+1]$ tenia más números menores o iguales a el antes en la lista lo que cumple el Invariante. En caso contrario es porque r sigue siendo el mayor número de repeticiones lo que sigue cumpliendo la Invariante.

Con esto terminamos de probar ambos ciclos, despues de esto j decrementa en 1 y lo seguirá haciendo hasta que eventualmente sea menor a 0 y el ciclo pare. Despues de esa ultima iteración los Invariantes se siguen cumpliendo ya que j toma el valor de -1, 1, toma el valor de j y r según lo demostrado siempre actualiza su valor correctamente, actualizando así también el valor m lo que también cumple la salida

* Teorema: la Invocación algoritmo3(A) devuelve el número m con más números menores o iguales a el en posiciones anteriores de A.

No es trivial ya que la demostración de la correctitud no está bien ya que no están bien planteados los invariantes ni se hizo la demostración del ciclo interno.

* Demostración: Es trivial a partir de la correctitud de los Invariantes I_0, I_1 e I_2.

Siempre se demuestran teoremas.

Punto 4

Teorema:

a) $7n^2 - 3n \in O(3n^2 \operatorname{sqrt} n)$

¿de dónde proviene esta expresión?

Demostración

Se procede a demostrar de manera directa:

$$\boxed{\exists n \geq n_0 .\ 7n^2 - 3n \leq c(3n^2 \operatorname{sqrt} n)}$$

$$\frac{7n^2 - 3n}{3n^2 n^{1/2}} \leq \frac{c(3n^2 \operatorname{sqrt} n)}{3n^2 \operatorname{sqrt} n}$$

$$\frac{7\cancel{n^2}}{3n^{\cancel{3/2}}} - \frac{\cancel{3}\cancel{n}}{\cancel{3}n^{\cancel{3/2}}} \leq c$$

$$\frac{7}{3n^{1/2}} - \frac{1}{n^{3/2}} \leq c$$

????

$n = 1$

$$\frac{7}{3(1)^{1/2}} - \frac{1}{(1)^{3/2}} \leq c$$

$$\frac{4}{3} \leq c$$

Es terrible la presentación de esta tarea!!!!

Con esto podemos ver que el valor mas grande que se puede tomar es 4/3, cuando n=1, y tomando en cuenta que n es denominador entre mas grande sea n, $n \geq 1$, tendera a 0
Por lo tanto con los testigos $c = 4/3$ y $n = 1$ demostramos que $7n^2 - 3n \in O(3n^2 \text{sqrt}\, n)$

b) Teorema: $2^n \in O(n!)$

Demostración

Se procede a demostrar de manera directa

$$\boxed{\forall n \geq n \rightarrow 2^n \leq cn!}$$

¿de dónde proviene esto?

$$\frac{2^n}{n!} \leq c \frac{\cancel{n!}}{\cancel{n!}}$$

$$\frac{2^n}{n!} \leq c$$

$n_0 = 0 \qquad 1 \leq c$

$$2^n \leq n!$$

$\circ\, n = 1$

$2^1 \leq 1! \rightarrow 2 \leq 1$

$\circ\, n = 2$

$4 \leq 2$

$\circ\, n = 3$

$8 \leq 6$

$\circ\, n = 4$

$16 \leq 24$

$\circ\, n = 5$

$32 \leq 120$

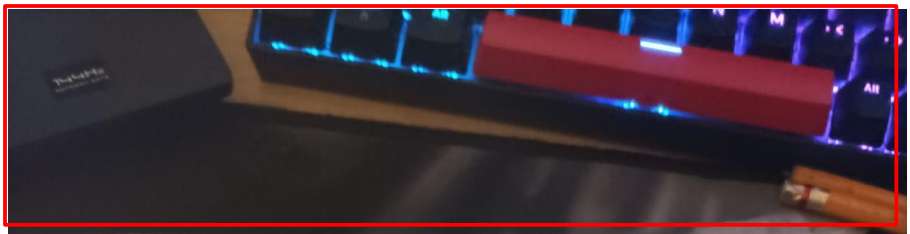
$\circ\, n = 6$

$64 \leq 720$

$\circ\, n = 0$

$1 \leq 1$

????



????

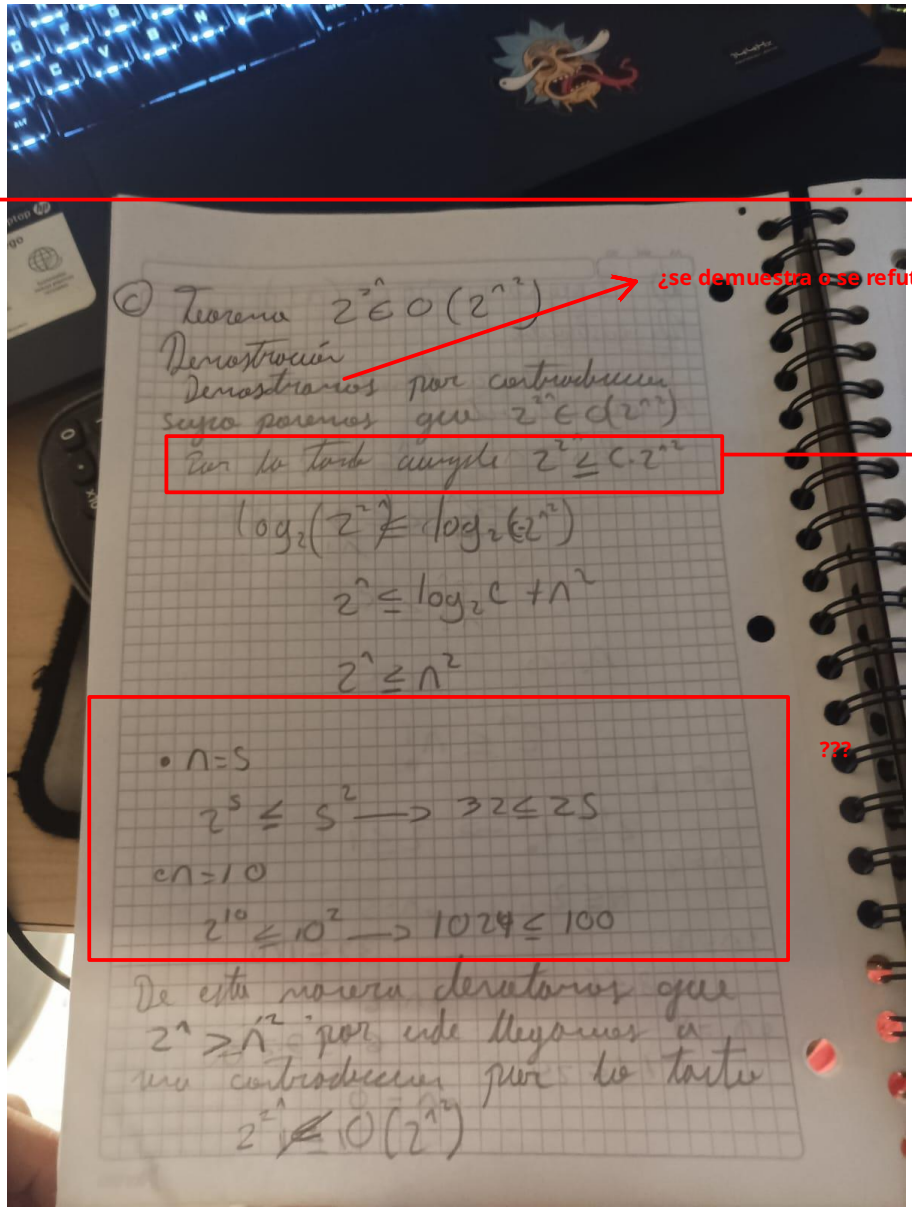
Con el testigo $c=1$ y $n_0=1$ qed
y con los datos analizados
podemos concluir que

con $n_0 \geq 4$ o $n_0=0$

$$2^n \in O(n!)$$

¿cómo se obtuvieron esos testigos?

????



c) Teorema $2^{2^n} \in O(2^{n^2})$

Demostración

Demostraremos por contradicción

Sea c podemos que $2^{2^n} \in O(2^{n^2})$

Por lo tanto cumple $2^{2^n} \leq c \cdot 2^{n^2}$

$\log_2(2^{2^n}) \leq \log_2(c \cdot 2^{n^2})$

$2^n \leq \log_2 c + n^2$

$2^n \leq n^2$

- $n = 5$

$2^5 \leq 5^2 \longrightarrow 32 \leq 25$

$c n = 10$

$2^{10} \leq 10^2 \longrightarrow 1024 \leq 100$

De esta manera derivamos que $2^n > n^2$ por ende llegamos a una contradicción por lo tanto

$2^{2^n} \not\in O(2^{n^2})$

¿se demuestra o se refuta?

¿de dónde se obtiene esto?

???

Integrantes: Nicolás Zapata y Juan José Aguilar

Esto debe ir al inicio del documento.

El documento está hecho a mano pero solo se observa un tipo de letra, ¿cómo funcionó el trabajo durante el desarrollo de la tarea?